

# ALGO VISION

Interactive Sorting & Searching Algorithm Visualizer

## PROJECT REPORT

|               |                                  |
|---------------|----------------------------------|
| Team Member 1 | _____                            |
| Team Member 2 | _____                            |
| Team Member 3 | _____                            |
| Team Member 4 | _____                            |
| College       | _____                            |
| Department    | Computer Science and Engineering |
| Hackathon     | _____                            |
| Academic Year | 2026–27                          |

# 1. Abstract

AlgoVision is an interactive web-based application designed to help students understand and visualize important Data Structures and Algorithms concepts. The project focuses on Merge Sort, Quick Sort, and Binary Search. Instead of displaying only the final answer, the website demonstrates algorithm operations step by step using visual animations. Users can enter numbers, select an algorithm, and view useful information such as steps, comparisons, execution time, and the final result.

## 2. Introduction

Data Structures and Algorithms are important concepts in Computer Science. Many students find sorting and searching algorithms difficult because they are often learned mainly through theory and code. AlgoVision provides a visual way to understand these algorithms by showing how values are compared, moved, divided, merged, or searched. This makes the learning process more interactive and easier for beginners.

## 3. Problem Statement

Students often have difficulty visualizing sorting and searching algorithms from textbooks or code alone. It can be difficult to understand how comparisons happen, how an array changes during execution, and how different algorithms behave. AlgoVision addresses this problem through an interactive visualizer.

## 4. Objectives

- Create an interactive algorithm visualization website.
- Demonstrate Merge Sort step by step.
- Demonstrate Quick Sort step by step.
- Demonstrate Binary Search on sorted data.
- Display algorithm steps and comparisons.
- Measure approximate execution time.
- Help students understand algorithm behavior visually.
- Provide a beginner-friendly interface.

## 5. Technologies Used

**HTML:** Provides the basic structure of the application.

**CSS:** Handles styling, layout, animations, and responsive design.

**JavaScript:** Implements algorithms and interactive functionality.

**Development Tools:** Visual Studio Code, web browser, and Git/GitHub if used.

**Design/Development Support:** Lovable, if used during project creation.

## 6. Algorithms Used

### 6.1 Merge Sort

Merge Sort uses the Divide and Conquer technique. It divides an array into smaller parts, sorts those parts, and merges them into a sorted array. Time complexity: Best  $O(n \log n)$ , Average  $O(n \log n)$ , Worst  $O(n \log n)$ .

## 6.2 Quick Sort

Quick Sort selects a pivot, partitions the array around the pivot, and recursively sorts the resulting parts. Time complexity: Best  $O(n \log n)$ , Average  $O(n \log n)$ , Worst  $O(n^2)$ .

## 6.3 Binary Search

Binary Search works on sorted data. It repeatedly checks the middle element and reduces the search range by half. Time complexity: Best  $O(1)$ , Average  $O(\log n)$ , Worst  $O(\log n)$ .

# 7. Main Features

- User input for numbers.
- Random array generation.
- Merge Sort visualization.
- Quick Sort visualization.
- Binary Search visualization.
- Step counter.
- Comparison counter.
- Execution-time measurement.
- Start, pause, resume, step, reset, and speed controls where implemented.
- Algorithm explanations and complexity information.

# 8. System Working

The basic workflow is:

**User Input → Algorithm Selection → Algorithm Execution → Visualization → Statistics → Final Result**

The user enters or generates an array, selects an algorithm, and starts the visualization. The application displays the algorithm's operations and calculates the relevant statistics. Finally, the sorted array or search result is displayed.

# 9. System Architecture

The project follows a simple flow: User → Input Numbers → Select Algorithm → Merge Sort / Quick Sort / Binary Search → Visualization → Steps, Comparisons, Execution Time → Final Result.

# 10. User Interface

The application includes a home/introduction section, input controls, algorithm selection, a visualization area, statistics, and algorithm information. The interface is designed to make the algorithm behavior easy to follow.

# 11. Advantages

- Easy to understand for beginners.
- Makes algorithms visually interactive.
- Shows algorithm operations step by step.

- Helps students understand comparisons.
- Provides performance information.
- Allows experimentation with different inputs.
- Useful for DAA learning and demonstrations.

## **12. Applications**

AlgoVision can be used by Computer Science students, programming laboratories, teachers for classroom demonstrations, and learners practicing Data Structures and Algorithms.

## **13. Results**

The developed website demonstrates Merge Sort, Quick Sort, and Binary Search using interactive visualizations. It accepts user input, performs the selected algorithm, displays the operations, and provides information such as steps, comparisons, execution time, and the final result.

## **14. Future Scope**

Future improvements can include Bubble Sort, Selection Sort, Insertion Sort, additional searching algorithms, graph and tree visualizers, side-by-side algorithm comparison, progress tracking, quizzes, advanced performance analysis, mobile support, and additional themes.

## **15. Conclusion**

AlgoVision is an interactive educational web application that makes sorting and searching algorithms easier to understand. By combining algorithm implementation with visualization and performance information, the project provides a simple and engaging learning experience. The project can be expanded with additional algorithms and advanced visualization features.

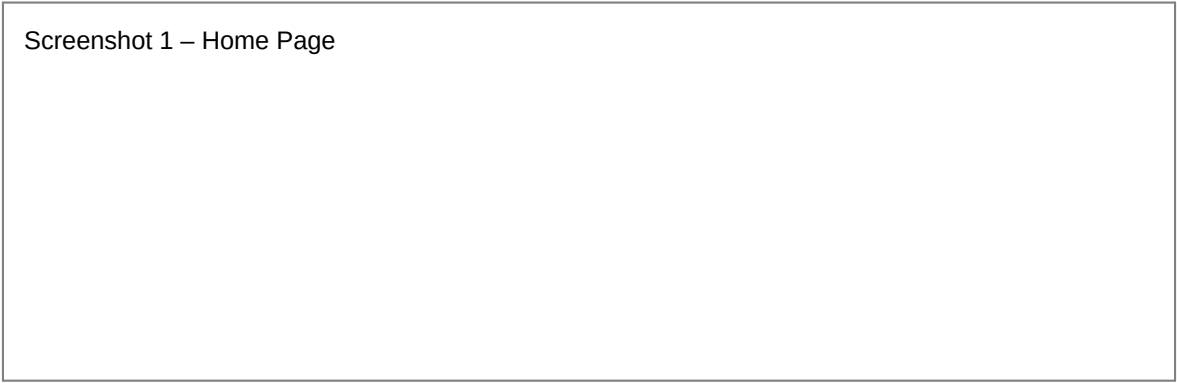
## **16. References**

1. Data Structures and Algorithms study materials.
2. JavaScript documentation.
3. HTML and CSS documentation.
4. Visual Studio Code documentation.
5. Algorithm textbooks and educational resources.
6. Online programming and algorithm learning resources.

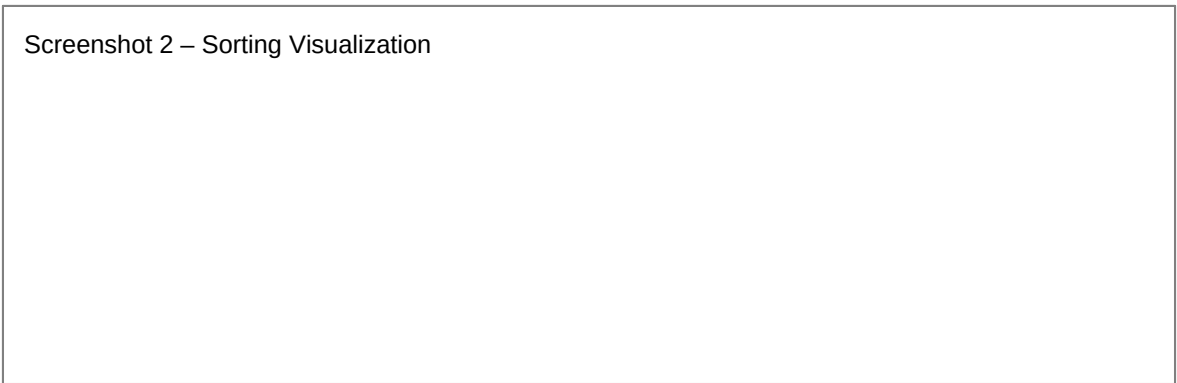
## 17. Project Screenshots

Add screenshots of the completed AlgoVision website in the spaces below.

Screenshot 1 – Home Page



Screenshot 2 – Sorting Visualization



Screenshot 3 – Binary Search / Results

